

How to Cancel Parallel Operations in C# with Examples

In this article, I am going to discuss **How to Cancel Parallel Operations in C#** with Examples. Please read our previous article, where we discussed [How to use the Maximum Degree of Parallelism in C#](#) with Examples.

How to Cancel Parallel Operations in C#?

Like Asynchronous Programming, we can use the Cancellation Token to cancel operations in Parallel Programming. We can use the same Cancellation Token here also. The ParallelOptions Class provides the option to cancel the parallel execution. If you right-click on the ParallelOptions class and select go to definition, then you will see the following. This class has one constructor and three properties.

```
namespace System.Threading.Tasks
{
    ...public class ParallelOptions
    {
        ...public ParallelOptions();

        ...public TaskScheduler TaskScheduler { get; set; }
        ...public int MaxDegreeOfParallelism { get; set; }
        ...public CancellationToken CancellationToken { get; set; }
    }
}
```

The ParallelOptions class in C# provides the following constructor which we can use to create an instance of ParallelOptions class.

1. **ParallelOptions():** It initializes a new instance of the ParallelOptions class.

The ParallelOptions Class Provides the following three properties.

1. **public TaskScheduler TaskScheduler {get; set;};** This property is used to get or set the TaskScheduler associated with the ParallelOptions instance. Setting this property to null indicates that the current scheduler should be used. It returns the task scheduler that is associated with this instance.
2. **public int MaxDegreeOfParallelism {get; set;};** This property is used to get or set the maximum number of concurrent tasks enabled by the ParallelOptions instance. It returns an integer that represents the maximum degree of parallelism.
3. **public CancellationToken CancellationToken {get; set;};** This property is used to get or set the CancellationToken associated with the ParallelOptions instance. It returns the token that is associated with the ParallelOptions instance.

Example without Cancelling the Parallel Operation in C#:

In the below example, we have set the degree of parallelism to 2 i.e. a maximum of two threads execute the methods parallelly. Here, we have not used Cancellation Token and hence the parallel execution will not be canceled.

```
using System;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create an instance of ParallelOptions class
            var parallelOptions = new ParallelOptions()
            {
                MaxDegreeOfParallelism = 2,
            };

            try
            {
                Stopwatch stopwatch = new Stopwatch();
                stopwatch.Start();
                //Passing ParallelOptions as the first parameter
                Parallel.Invoke(
                    parallelOptions,
                    () => DoSomeTask(1),
                    () => DoSomeTask(2),
                    () => DoSomeTask(3),
                    () => DoSomeTask(4),
                    () => DoSomeTask(5),
                    () => DoSomeTask(6),
                    () => DoSomeTask(7)
                );
                stopwatch.Stop();
                Console.WriteLine($"Time Taken to Execute all the Methods :
{stopwatch.ElapsedMilliseconds/1000.0} Seconds");
            }
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
            }

            Console.ReadLine();
        }
        static void DoSomeTask(int number)
        {
            Console.WriteLine($"DoSomeTask {number} started by Thread
{Thread.CurrentThread.ManagedThreadId}");
            //Sleep for 2 seconds
        }
    }
}
```

```

        Thread.Sleep(TimeSpan.FromSeconds(2));
        Console.WriteLine($"DoSomeTask {number} completed by Thread
{Thread.CurrentThread.ManagedThreadId}");
    }
}
}

```

Output:

```

DoSomeTask 1 started by Thread 1
DoSomeTask 2 started by Thread 3
DoSomeTask 2 completed by Thread 3
DoSomeTask 1 completed by Thread 1
DoSomeTask 3 started by Thread 3
DoSomeTask 4 started by Thread 1
DoSomeTask 4 completed by Thread 1
DoSomeTask 3 completed by Thread 3
DoSomeTask 6 started by Thread 3
DoSomeTask 5 started by Thread 1
DoSomeTask 5 completed by Thread 1
DoSomeTask 6 completed by Thread 3
DoSomeTask 7 started by Thread 1
DoSomeTask 7 completed by Thread 1
Time Taken to Execute all the Methods : 8.086 Seconds

```

If you observe the output maximum of 2 threads is there to execute code parallelly. Further notice it took approximately little more than 8 seconds to complete the execution. The time duration may vary on your machine. Now, what we are going to do is, we will cancel the parallel execution after 5 seconds.

How to Cancel Parallel Operations in C#?

In order to Cancel the Parallel Operations in C#, first, we need to create an instance of `ParallelOptions` class and then we need to create an instance of `CancellationTokenSource` and then we need to set the `CancellationToken` properties of `ParallelOptions` instance to the token of the `CancellationTokenSource` instance. The following image shows the syntax to use `CancellationToken` to cancel the parallel execution in C#.

```

//Create an Instance of CancellationTokensource
var CTS = new CancellationTokensource();

//Set when the token is going to cancel the parallel execution
CTS.CancelAfter(TimeSpan.FromSeconds(5));

//Create an instance of ParallelOptions class
var parallelOptions = new ParallelOptions()
{
    //Set the CancellationToken value
    CancellationToken = CTS.Token
};

```

Example to Understand How to Cancel Parallel Operations in C#:

The following is the complete code example. In the below example, we are canceling the Parallel Execution after 4 seconds. In asynchronous programming, we already discussed that when the token is canceled it throws an exception, so we have written the try-catch block here to handle that exception. As always, it is a good programming practice the dispose of the token and set its value to null in the finally block.

```

using System;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of CancellationTokensource
            var CTS = new CancellationTokensource();
            //Set when the token is going to cancel the parallel execution
            CTS.CancelAfter(TimeSpan.FromSeconds(5));
            //Create an instance of ParallelOptions class
            var parallelOptions = new ParallelOptions()
            {
                MaxDegreeOfParallelism = 2,
                //Set the CancellationToken value
                CancellationToken = CTS.Token
            };
            try
            {
                Stopwatch stopwatch = new Stopwatch();
                stopwatch.Start();
            }
            catch { }
            finally
            {
                CTS.Dispose();
                CTS.Token = null;
            }
        }
    }
}

```

```

//Passing ParallelOptions as the first parameter
Parallel.Invoke(
    parallelOptions,
    () => DoSomeTask(1),
    () => DoSomeTask(2),
    () => DoSomeTask(3),
    () => DoSomeTask(4),
    () => DoSomeTask(5),
    () => DoSomeTask(6),
    () => DoSomeTask(7)
);
stopwatch.Stop();
Console.WriteLine($"Time Taken to Execute all the Methods :
{stopwatch.ElapsedMilliseconds/1000.0} Seconds");
}
//When the token cancelled, it will throw an exception
catch(Exception ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    //Finally dispose the CancellationTokensource and set its value to null
    CTS.Dispose();
    CTS = null;
}
Console.ReadLine();
}
static void DoSomeTask(int number)
{
    Console.WriteLine($"DoSomeTask {number} started by Thread
{Thread.CurrentThread.ManagedThreadId}");
    //Sleep for 2 seconds
    Thread.Sleep(TimeSpan.FromSeconds(2));
    Console.WriteLine($"DoSomeTask {number} completed by Thread
{Thread.CurrentThread.ManagedThreadId}");
}
}
}

```

Output:

```
DoSomeTask 1 started by Thread 1
DoSomeTask 2 started by Thread 4
DoSomeTask 2 completed by Thread 4
DoSomeTask 1 completed by Thread 1
DoSomeTask 3 started by Thread 4
DoSomeTask 4 started by Thread 1
DoSomeTask 4 completed by Thread 1
DoSomeTask 3 completed by Thread 4
The operation was canceled.
```

When you run the application, please observe the output carefully. Here, it started the execution parallelly by using two threads. It will continue execution until the token is canceled i.e. for 4 seconds. As soon as the token is canceled, the parallel execution stopped and it will throw the token canceled exception which is handled by the catch block, and in the catch block, we just print the exception message that is what you see in the last statement of the output.

Canceling Parallel Operation Example using Parallel Foreach Loop in C#:

In the below example, the collection contains 20 elements which means the Parallel Foreach loop will execute 20 times. And here we set the `MaxDegreeOfParallelism` property to 2 which means a maximum of two threads will execute the loop parallelly. Further, we have delayed the execution for 1 second. Then we set the Cancellation Token duration to 5 seconds i.e. after 5 seconds the cancellation token will cancel the parallel execution.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of CancellationTokenSource
            var CTS = new CancellationTokenSource();
            //Set when the token is going to cancel the parallel execution
            CTS.CancelAfter(TimeSpan.FromSeconds(5));
            //Create an instance of ParallelOptions class
            var parallelOptions = new ParallelOptions()
            {
                MaxDegreeOfParallelism = 2,
                //Set the CancellationToken value
                CancellationToken = CTS.Token
            };
            try
            {
                List<int> integerList = Enumerable.Range(0, 20).ToList();
                Parallel.ForEach(integerList, parallelOptions, i =>
```

```

        {
            Thread.Sleep(TimeSpan.FromSeconds(1));
            Console.WriteLine($"Value of i = {i}, thread = {Thread.CurrentThread.ManagedThreadId}");
        });
    }
    //When the token canceled, it will throw an exception
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
    finally
    {
        //Finally dispose the CancellationTokenSource and set its value to null
        CTS.Dispose();
        CTS = null;
    }
    Console.ReadLine();
}
}
}

```

Output:

```

Value of i = 10, thread = 4
Value of i = 0, thread = 1
Value of i = 11, thread = 4
Value of i = 1, thread = 1
Value of i = 2, thread = 1
Value of i = 12, thread = 4
Value of i = 3, thread = 1
Value of i = 13, thread = 5
Value of i = 4, thread = 1
Value of i = 14, thread = 5
The operation was canceled.

```

Canceling Parallel Operation Execution Example using Parallel For Loop in C#:

In the below example, the Parallel For loop will execute 20 times. Here we set the `MaxDegreeOfParallelism` property to 2 which means that a maximum of two threads will execute the for loop parallelly. Further, we have delayed the execution for 1 second intentionally so that we have a chance to cancel the execution after a certain period of time. Then we set the Cancellation Token duration to 5 seconds i.e. after 5 seconds the cancellation token will cancel the parallel for loop execution by throwing an exception which is handled by the catch block and finally, in the finally block we have to dispose of the token and set its value to null.

```

using System;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo

```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of CancellationTokenSource
            var CTS = new CancellationTokenSource();
            //Set when the token is going to cancel the parallel execution
            CTS.CancelAfter(TimeSpan.FromSeconds(5));
            //Create an instance of ParallelOptions class
            var parallelOptions = new ParallelOptions()
            {
                MaxDegreeOfParallelism = 2,
                //Set the CancellationToken value
                CancellationToken = CTS.Token
            };
            try
            {
                Parallel.For(1, 21, parallelOptions, i => {
                    Thread.Sleep(TimeSpan.FromSeconds(1));
                    Console.WriteLine($"Value of i = {i}, thread =
{Thread.CurrentThread.ManagedThreadId}");
                });
            }
            //When the token canceled, it will throw an exception
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
            }
            finally
            {
                //Finally dispose the CancellationTokenSource and set its value to null
                CTS.Dispose();
                CTS = null;
            }
            Console.ReadLine();
        }
    }
}

```

Output:

```
Value of i = 1, thread = 1  
Value of i = 11, thread = 4  
Value of i = 2, thread = 1  
Value of i = 12, thread = 7  
Value of i = 3, thread = 1  
Value of i = 13, thread = 7  
Value of i = 4, thread = 1  
Value of i = 14, thread = 4  
Value of i = 5, thread = 1  
Value of i = 15, thread = 4  
The operation was canceled.
```

Note: Everything that we learn in asynchronous programming regarding the Cancellation Token Source and Cancellation Token is applicable to parallel programming.

In the next article, I am going to discuss a few important things i.e. [Atomic Methods, Thread Safety, and Race Conditions in C#](#) Parallel Programming with Examples. Here, in this article, I try to explain How to use **How to Cancel Parallel Operations in C#** with Examples. I hope you enjoy this How to Cancel Parallel Operations in C# with Examples.